Snake Out of Hole

1. Game Introduction

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the Dofbot corresponding container:

```
./Docker_Ros.sh
```

Then enter the following command in the docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address plus :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

```
Game instructions:
1) First close the main program;
2) Start a command line window as the inverse solution service, running
continuously.
ros2 run dofbot_info kinemarics_dofbot
3) Snake out of hole game path:
/root/Dofbot_2D/src/dofbot_snake_follow/scripts/引蛇出洞.ipynb
```

(1) Start the code block, display the interface as shown in the figure, freely click the color button to select colors, the selected color is displayed in the middle above the image,

(2) When the selected color appears in the field of view, the robotic arm will estimate the block's position based on the block's area in the image.

(3) When the area becomes small, the robotic arm drives forward until the front end completes the grabbing action. When the area becomes large, it retreats and shakes its head in a fearful state.

(4) If the color appearing on the screen is not the selected color, the robotic arm retreats and shakes its head.

Note: Here use color blocks to control the robotic arm back and forth, then slowly move away to trigger. This case requires multiple attempts

2. Code Design

- Use polygon approximation method to obtain the area of color objects in the field of view

```
for i, cnt in enumerate(contours):
    # Calculate the polygon's moments
    mm = cv.moments(cnt)
    cx = mm['m10'] / mm['m00']
    cy = mm['m01'] / mm['m00']
    # Calculate the area of the contour
    area = cv.contourArea(cnt)
```

- Get current posture through forward kinematics

```
def get_Posture(self):
    '''
    Publish joint angles, get position
    '''
    self.read_joint()
    # Wait for the server to start
    self.client.wait_for_service()
    # Create a message pack
    request = kinemaricsRequest()
    ...   ...
```

- Estimate position based on area size

```
# Estimate the block's position relative to the camera
distance = 27.05 * math.pow(area, -0.51) - 0.2
# Estimate the block's position in the base coordinate system
target_dist = distance + self.Posture[1]
```

- Get the angles each joint needs to rotate through inverse kinematics

```
def snake_run(self, point_y):
    '''
    Publish position request, get joint rotation angle
    '''
    pass
```

- Inverse solution design

  When we play snake out of hole, we need to control the end posture of the robotic arm, always facing forward. When solving for the inverse solution, just fix the end posture of the robotic arm.

```
/////////////// Snake out of hole  ///////////////
if (request.tar_z >= 0.2) {
      x=request.tar_x;
      y=request.tar_y;
      z=request.tar_z;
      Roll= -90;
}
Code path dofbot_ws/src/dofbot_info/src/dofbot_server.cpp
```

For detailed code, please check snake_ctrl.py; snake_move.py; snake_target.py in the /Dofbot_2D/src/dofbot_snake_follow/scripts directory